Machine Learning 2 (APM_50441_EP/DSAIB - 2026-2027). Lab1

The objectives of the lab

This lab illustrates the first lecture (*Setting, Risk Estimation... and Time Series*). Its goals are:

- to estimate the risk of a predictor and to measure the uncertainty of this estimation (hold-out, cross validation, comparison of two models, selection rules),
- to take into account a difference between the training and the real-world proportions, and a cost depending on the prediction (weights, Bayes rule),
- to compare hyperparameter search strategies with a fixed budget, and to see how much the selection itself can be overfitted,
- to validate a forecast correctly when the data are a time series.

The optional *To go further* part, after the core, can be skipped at first.

Use of AI. AI assistants (e.g. the ones built into Colab or VS Code) are allowed, but you must be able to explain every line you use, and answer the *predict before you run* questions before running the code.

Core (about 2 hours)

Exercise 1 (Risk estimation and model comparison)

The *risk* of a predictor f is its average loss on a new observation, $\mathcal{R}(f) = \mathbb{E}[\ell(Y, f(X))]$. It is unknown and must be estimated from data that have not been used to train f.

1. *Predict before you run.* For the 1-nearest neighbor, the 15-nearest neighbors and the logistic regression, what do you expect for the accuracy on the training set compared with the accuracy on the test set? Check it.

   A single test set gives a single, noisy, estimate. To see how noisy, we can repeat the *hold-out* procedure: split the training set into a training part and a validation part many times.

2. Estimate the accuracy of the logistic regression with 20 random hold-out splits ($80\%/20\%$) of the training set, and with a 5-fold cross validation repeated 4 times (`StratifiedKFold(n_splits=5, shuffle=True, random_state=i)`). Compare the spread of the individual estimates, and of the averages.

   The standard deviation of the fold scores describes the variability of the accuracy of *one* fitted model on *one* validation set; the standard error (the standard deviation divided by $\sqrt{V}$) describes the variability of their *mean*, under an independence assumption that does not hold (the training parts of the folds overlap).

3. Compare the logistic regression and the 15-nearest neighbors on the *same* folds (5-fold CV repeated 4 times). Compute the paired differences, and compare a naive t-test, which assumes independent differences, with the *corrected resampled t-test* of Nadeau and Bengio, which inflates the variance of the mean difference from s^2/J to $(1/J + n_{\text{val}}/n_{\text{train}})\, s^2$.

4. Choose the number of neighbors k among the values below with a 5-fold cross validation, with three rules: the best average accuracy, the simplest model (largest k) whose average is within one standard error of the best one (*one standard error rule*), and the best *worst fold* (a pessimistic, quantile-based rule in the spirit of the PAC approach of the lecture). Evaluate the selected models once on the test set.

Exercise 2 (Unbalanced data, weights and costs)

In our dataset, about one third of the loans are `Risk`, whereas we assume that in the real world the proportion

of `Risk` is $\pi_{\text{Risk}} = 0.02$. If the conditional law of X given Y is the same in both worlds, an expectation in the real world is a *weighted* expectation in the training world, with a weight $w_y = \pi_y / \pi_y^{\text{train}}$ depending only on the class:

$$\mathbb{E}_{\text{real}}[\ell(Y, f(X))] = \mathbb{E}_{\text{train}}[w_Y \, \ell(Y, f(X))].$$

1. Write a function `custom_metric(y_true, y_pred)` computing the real-world accuracy (the weighted accuracy) and the corresponding scorer. What is the real-world accuracy of the logistic regression? Of a classifier that always predicts `No Risk`?

   Two ways of adapting the predictor are seen in the lecture: training with the weights (*weighted loss*), or keeping the probabilities estimated in the training world and changing the decision (*plug-in Bayes rule*): predict `Risk` when $w_{\text{Risk}}\,\hat{p}(\text{Risk}|x) > w_{\text{No Risk}}\,\hat{p}(\text{No Risk}|x)$.

   To send weights to a step of a pipeline, `scikit-learn` uses *metadata routing*: each step declares whether it uses them.

2. Train the logistic regression with the weights (only the classifier should use them), and implement the plug-in decision. Compare their real-world accuracy on the test set.

   The accuracy is not the right criterion here: the cost of an error depends on the error and on the amount of the loan. We use the following cost model (the opposite of a gain):

   - $5 + 0.6 \times$ LoanAmount if we grant a loan that defaults (`Risk` predicted `No Risk`),
   - $1 - 0.05 \times$ LoanAmount if we grant a good loan (`No Risk` predicted `No Risk`): a benefit of $0.05 \times$ LoanAmount after a fixed cost of 1,
   - 1 if we refuse the loan.

   The cost model, the weighted cost metric and the corresponding scorers are gathered in `apm_credit.py`. The scorers return the *gain* (the opposite of the cost), and need `LoanAmount` as metadata.

3. *Predict before you run.* Write the Bayes rule for this cost: for a loan of amount a and an estimated probability p of `Risk`, when should we refuse it? Compute the threshold on p for a loan of 5000, and compare it with the threshold $1/2$ of `predict`.

4. Compare, with a 5-fold cross validation of the gain on the training set, the original logistic regression, the logistic regression trained with the cost weights `compute_cost_weights(y, LoanAmount)`, the Bayes decision on the probabilities of the original one (`cost_scorer_from_proba`), and a threshold tuned by cross validation (`TunedThresholdClassifierCV`). Which one would you choose? Evaluate it on the test set.

Exercise 3 (Hyperparameter optimization with a fixed budget)

We now tune a `HistGradientBoostingClassifier` (a model of Lecture 3, fast to train) for the cost-based gain of the Bayes decision. The lecture presents several strategies: an exhaustive grid, random search, *successive halving* (many candidates evaluated on small budgets, the best ones promoted to larger budgets), and Bayesian optimization (here Optuna's Tree-structured Parzen Estimator, TPE). We are going to compare them with roughly the same budget of 20 full 3-fold cross validations.

Note: `HistGradientBoostingClassifier` already uses several cores (we limited them to 2 at the beginning of the lab); running several cross validations in parallel (`n_jobs`) on top of it oversubscribes them and is often *slower*, so we keep `n_jobs=None` here.

1. Run a grid search on the 4×5 grid `grid`, a random search with 20 draws from `distributions`, a successive halving search (`HalvingRandomSearchCV` with 60 candidates, `factor=3` and `min_resources='exhaust'`, the resource being the number of training samples) and an Optuna study with 20 trials of a TPE sampler, all with the gain of the Bayes decision (`cost_scorer_from_proba`) as score. Compare their best CV gains, their running times, and the gains of the selected models on the test set.

   The best CV score of a search is the maximum of noisy estimates: it is optimistic, and the more candidates are tried, the more optimistic it is (*overfitting the validation set*). To see this *selection bias* alone, we select the configuration with a first cross validation (folds A), and re-estimate the selected configuration with an independent second cross validation (folds B, another shuffle of the same data): the configuration does not change, only the noise does. As in the lecture, the phenomenon is much clearer with a small training set: we use 400 loans of the training set.

2. Draw 60 random configurations from `distributions`. For each of them, compute the 3-fold CV gain on the small training set with two different shuffles (A and B). As a function of the number m of configurations tried, plot the best CV gain on A (the reported one) and the CV gain on B of the configuration selected on A. Comment.

Exercise 4 (**Validation of a time series forecast**)

We forecast the monthly CO_2 concentration at Mauna Loa (1958–2001, bundled with `statsmodels`) **one year ahead**: at the end of month t, we want y_{t+12}. Only the values up to time t can be used, so the features of the target y_s can only use $y_{s-12}, y_{s-13}, \ldots$ (the *lags* larger than the horizon), the calendar (month) and the time.

The last 10 years (1992–2001) are kept as a test period.

1. The *seasonal naive* forecast predicts y_{t+12} by y_t, i.e. $\hat{y}_s = y_{s-12}$ (`lag_12`). Compute its mean absolute error (MAE) on the test period. Then fit a ridge regression and a `HistGradientBoostingRegressor` on the `features`, and compute their test MAE.

2. *Predict before you run.* We want to choose between these models with the training period only. Compare the MAE estimated by a shuffled 5-fold cross validation, by `TimeSeriesSplit(n_splits=5, test_size=24, gap=H)` (each validation block comes after its training part, with a gap of at least the horizon), and the test MAE. Which estimation is reliable?

3. Trees cannot extrapolate: the gradient boosting predicts values in the range of its training targets. *Stationarize* the series by forecasting the seasonal difference $y_{t+12} - y_t$ (i.e. predict `co2 - lag_12` and add `lag_12` back), and compare the test MAE of both models.

To go further

The following blocks are optional and independent: each of them only uses the objects defined in the core part. Their difficulty is marked by stars: ⋆ direct extension, ⋆⋆ new method, ⋆⋆⋆ longer or research-flavored.

Block	Lecture	Stars
Bootstrap estimates: .632 and .632+	Risk estimation	⋆
The number of folds and leave-one-out	Risk estimation	⋆
Penalized criteria: AIC and BIC	Risk correction	⋆
Nested cross validation	CV and test	⋆⋆
McNemar's test and the lucky winner	Comparison of means	⋆⋆
Resampling instead of weights	Unbalanced data	⋆⋆
Calibration and cost-based decisions	Weighted loss	⋆⋆
The geometry of cost-sensitive decisions	Weighted loss	⋆⋆
Pruning and other samplers in Optuna	AutoML	⋆⋆
AutoML frameworks and stacking	AutoML	⋆⋆
A tabular foundation model: AutoML without search	AutoML, meta-learning	⋆⋆
STL decomposition	Time series	⋆
ARIMA models	Time series	⋆⋆
Prediction intervals	Time series	⋆⋆
A small neural forecaster	Time series	⋆⋆

Block	Lecture	Stars
Zero-shot forecasting with a foundation model	Time series	⋆⋆⋆

Exercise 5 (**Bootstrap estimates: .632 and .632+** ⋆)
The bootstrap draws n observations with replacement: about 63.2% of the distinct observations are drawn, the others (*out-of-bag*, OOB) can be used for validation. The OOB error is pessimistic (each model is trained on about 63% of the data) while the training error is optimistic; the *.632 estimate* $0.368\,\widehat{\mathrm{err}}_{\mathsf{train}} + 0.632\,\widehat{\mathrm{err}}_{\mathsf{OOB}}$ mixes them. When the model overfits a lot, the *.632+ estimate* gives more weight to the OOB error, with a weight depending on the *relative overfitting rate* $R = (\widehat{\mathrm{err}}_{\mathsf{OOB}} - \widehat{\mathrm{err}}_{\mathsf{train}})/(\gamma - \widehat{\mathrm{err}}_{\mathsf{train}})$, where γ is the *no-information error rate* (the error when the labels are independent of the features): $\hat{w} = 0.632/(1 - 0.368R)$.

- For the 1-NN, the 15-NN and the logistic regression, compute the training error, the OOB error (with 50 bootstrap samples), the .632 and .632+ estimates, and compare them with the test error.

Exercise 6 (**The number of folds and leave-one-out** ⋆)
With V folds, each model is trained on a fraction $(V-1)/V$ of the data: a small V gives a pessimistic estimate (less training data), a large V gives more correlated models. For a linear regression, the leave-one-out (LOO, $V = n$) error has a closed form: the residual of the observation i when it is left out is $(Y_i - \hat{Y}_i)/(1 - h_{ii})$, where h_{ii} is the i-th diagonal element of the *hat matrix* $\mathbb{X}(\mathbb{X}^\top\mathbb{X})^{-1}\mathbb{X}^\top$.

- (a) For the logistic regression, compute the V-fold CV accuracy for $V \in \{2, 5, 10, 20\}$, repeated 5 times with different shuffles: how do the average and the spread across repetitions depend on V?
  (b) Predict `LoanAmount` from the other numerical features with a linear regression on 500 loans: check the closed form of the LOO error against 500 fits.

Exercise 7 (**Penalized criteria: AIC and BIC** ⋆)
Instead of estimating the risk on new data, one can correct the training risk by a penalty: for a model with d parameters fitted by maximum likelihood, $\mathrm{AIC} = -2\log\mathcal{L} + 2d$ and $\mathrm{BIC} = -2\log\mathcal{L} + d\log n$ (the smaller, the better). They only need one fit per model.

- Fit logistic regressions (with `statsmodels`) on nested sets of features: the numerical features only, then adding the categorical features one by one (in the order of the columns). Compare the models selected by AIC, by BIC, and by a 5-fold CV of the log loss.

Exercise 8 (**Nested cross validation** ⋆⋆)
The CV score of the selected k is optimistic, as it is the best of several candidates evaluated on the same folds. *Nested* cross validation estimates the performance of the whole procedure, selection included: an outer cross validation splits the training set, and on each outer training part an inner cross validation (`GridSearchCV`) selects k; the selected model is evaluated on the outer validation part.

- Estimate by nested cross validation the accuracy of the selection of k (best mean rule) among `n_neighbors_values`, and compare it with the best CV accuracy of the core part. Which k is selected in each outer fold?

Exercise 9 (**McNemar's test and the lucky winner** ⋆⋆)
On a single test set, two classifiers can be compared with McNemar's test, which only looks at the observations on which they disagree: if they were equally good, each of them would be the only one right on those observations with probability $1/2$. When *many* models are compared, the best one is partly the luckiest one: this is the *lucky winner* of the lecture.

- (a) Apply McNemar's test to the logistic regression and the 15-NN on the test set.

(b) Simulate the lucky winner: draw 20 "models" that predict `Risk` at random with the same rate as the logistic regression, keep the best one on the first half of the test set, and evaluate it on the second half. Repeat 200 times.

Exercise 10 (**Resampling instead of weights** $\star\star$)

Instead of weights, the proportions of the training set can be changed by *resampling*: undersampling the majority class, oversampling the minority class, or generating synthetic minority observations (SMOTE). Here the real world has *fewer* `Risk` than the training set, so the resampling goes the other way: the `Risk` class is undersampled (or the `No Risk` class oversampled) to reach 2% of `Risk`.

- Train the logistic regression on a training set resampled to the real proportions, by undersampling `Risk` (drawing without replacement) and by oversampling `No Risk` (drawing with replacement), and compare their real-world accuracy and their gain (with `cost_scorer`) with the weighted training. Which drawback of undersampling appears?

  (The `imbalanced-learn` package provides these resamplers, as well as SMOTE, but it is not compatible with the metadata routing of `scikit-learn` that we enabled, so we need to resample by hand with `polars`.)

Exercise 11 (**Calibration and cost-based decisions** $\star\star$)

The Bayes decision uses the estimated probabilities as if they were the true conditional probabilities. A classifier is *calibrated* when, among the loans to which it gives a probability p of `Risk`, a proportion p are indeed `Risk`. This can be checked with a reliability diagram and with *proper scores* (Brier score, log loss), and improved by learning a monotone mapping of the scores by cross validation (`CalibratedClassifierCV`).

- Compare the calibration of the logistic regression, of the 15-NN and of the gradient boosting selected by the grid search. Calibrate the 15-NN (isotonic) and the gradient boosting (sigmoid): do their Bayes decisions gain more?

Exercise 12 (**The geometry of cost-sensitive decisions** $\star\star$)

Each threshold on the probability of `Risk` gives a point of the ROC curve: its false positive rate (good loans refused) and its true positive rate (defaulting loans refused). If the costs did not depend on the amount, the regret of a threshold would be $\pi_{\text{No Risk}}\, c_{FP}\, \text{FPR} + \pi_{\text{Risk}}\, c_{FN}\, (1 - \text{TPR})$: the thresholds with the same regret lie on a line of slope $\pi_{\text{No Risk}} c_{FP} / (\pi_{\text{Risk}} c_{FN})$ (an *iso-cost line*), and the best threshold is where such a line touches the ROC curve.

- Plot the ROC curve of the logistic regression on the test set with the iso-cost line for a loan of average amount, and the points of `predict` (threshold $1/2$), of the tuned threshold and of the Bayes rule.

Exercise 13 (**Pruning and other samplers in Optuna** $\star\star$)

Successive halving can also be used *inside* a Bayesian optimization: each trial reports intermediate scores (here after $20, 40, \ldots, 100$ boosting iterations, thanks to `warm_start`), and a *pruner* stops the unpromising trials early (Hyperband runs several successive halving brackets). Optuna also offers other samplers, e.g. CMA-ES, an evolution strategy (the *evolutionary* approach of the lecture).

- Run 40 trials of TPE with a `HyperbandPruner` on the learning rate and the number of leaves, reporting the 3-fold CV gain after $20, 40, \ldots, 100$ iterations. How many trials are pruned, and how much time is saved compared with the 20 unpruned trials of the core part? Run then 20 trials with the `CmaEsSampler`.

Exercise 14 (**AutoML frameworks and stacking** $\star\star$)

AutoML frameworks search over pipelines (models and hyperparameters) with a time budget. `FLAML` uses a cost-aware search; AutoGluon (heavier: on Colab, or locally in a separate environment, see the AutoGluon

notebook of Lab 3) trains many models and *stacks* them. Stacking can also be done by hand: a final model learns how to combine the cross-validated predictions of several base models.

Our gain is not a standard metric, so we let the frameworks optimize the log loss (good probabilities), and take the Bayes decision on their probabilities.

- Run FLAML with a time budget of 60 seconds (log loss), and build a `StackingClassifier` of the logistic regression, the 15-NN and the tuned gradient boosting. Compare the gains of their Bayes decisions with the ones of the core part. (Optional, on Colab or in a separate local environment: AutoGluon with `presets='medium_quality'` and `time_limit=120`.)

Exercise 15 (**A tabular foundation model: AutoML without search** $\star\star$)

Meta-learning goes one step further than AutoML: a model pretrained on millions of synthetic datasets (here TabICL v2, Lecture 3) predicts the labels of the test set *in context*, from the training set, without any training nor hyperparameter search. It runs on a CPU for a dataset of this size, but is much faster on a GPU.

- Compute the gain of the Bayes decision with the probabilities of `TabICLClassifier`, and compare it with the tuned gradient boosting.

Exercise 16 (**STL decomposition** $\star$)

STL (Seasonal-Trend decomposition using LOESS) splits a series into a trend, a seasonal component and a remainder with local regressions. The *strength* of the seasonality, $\max(0, 1 - \mathrm{Var}(R)/\mathrm{Var}(S + R))$, measures how much of the detrended variability is seasonal.

- Decompose the monthly CO_2 series with `statsmodels`' `STL` (period 12), plot the components and compute the strengths of the trend and of the seasonality.

Exercise 17 (**ARIMA models** $\star\star$)

The statistical approach models the dependence explicitly: a seasonal ARIMA $(p, d, q) \times (P, D, Q)_{12}$ differences the series (d times, and D times at lag 12) and models the result as an ARMA process. Once fitted on the training period, the same model can be *applied* to new data without refitting, to forecast from each origin.

- Fit a SARIMA$(1, 1, 1) \times (0, 1, 1)_{12}$ on the training period (the *airline model*). For each origin of the test period, forecast 12 months ahead (with `results.apply` on the series up to the origin), and compare the MAE with the ridge regression of the core part.

Exercise 18 (**Prediction intervals** $\star\star$)

A forecast is more useful with its uncertainty. Two simple ways to get an 80% interval: fit two quantile regressions (levels 0.1 and 0.9, with the *pinball* loss), or use *split conformal prediction*: compute the absolute residuals of a model on a calibration period that follows its training period, and add $\pm$ their 80% quantile to the forecasts.

- On the seasonal difference target, build 80% intervals with two quantile gradient boostings, and with split conformal prediction around the ridge regression (calibration period: 1986–1991, after a training period ending in 1985). Compare their coverage and their average width on the test period.

Exercise 19 (**A small neural forecaster** $\star\star$)

The deep learning approach of the lecture learns the forecast function with a neural network. On such a small series, a small multilayer perceptron is enough to see the main points: a neural network does not extrapolate either, so its inputs *and* its target must be stationary; the inputs must be standardized; and the training must be stopped before it overfits.

- Build stationary inputs: the differences `lag_k - lag_12` for $k = 13, \ldots, 24$ and the Fourier terms. Train with PyTorch a perceptron with one hidden layer of 32 units on these standardized inputs, predicting

the standardized seasonal difference, with Adam and early stopping on the last 3 years of the training period. Compare its test MAE with the ridge regression. (What happens with the raw `features`, which contain the level of the series?)

Exercise 20 (**Zero-shot forecasting with a foundation model** $\star\star\star$)

Time series foundation models are pretrained on very many (real and synthetic) series and forecast a new series *zero-shot*, from its past only. Chronos-2 (Amazon, 2025) is one of them; it outputs quantiles. It runs on a CPU for a few series (a GPU is faster).

- For each January of the test period (1992 to 2001), give Chronos-2 the series up to the previous December and forecast the next 12 months. Compare the MAE of the 12-months-ahead forecasts (the last month of each forecast) with the ridge regression on the same months, and the coverage of its 80% interval.

Use of AI. Artificial intelligence tools have been used to revise the content of this lab. The main contributor takes full responsibility for this content.

License. The texts of this lab are under the Creative Commons Attribution-ShareAlike 4.0 license (CC BY-SA 4.0), and its code under the MIT license; the datasets keep their own licenses. Main contributor: E. Le Pennec. Contributors: S. Boucheron, A. Dieuleveut, A.K. Fermin, S. Gadat, S. Gaiffas, B. Goujaud, A. Guilloux, Ch. Keribin, Th. Klein, E. Matzner, J. Read, M. Sangnier, E. Scornet.